

Technische Dokumentation Kühlschrankmanager KM3003

Samuel Brunner

November 20, 2024

Contents

1	Einleitung	3
2	Plattform und Systemarchitektur	3
3	Software	4
3.1	Verwendete Bibliotheken und Frameworks	4
3.1.1	Graphical User Interface	4
3.1.2	Datenbank	5
3.2	Datenbankstruktur	6
3.3	Testing	7
4	Deployment	7
5	Herausforderung bei der Implementierung	7
6	Ausblick und Verbesserungsvorschläge	7
7	Inline documetation anhängen	7

1 Einleitung

Der Kühltischmanager im Vertretungszimmer der Fachschaft ersetzt die Strichliste, welche früher am Kühltisch hing, und die Getränkekäufe der Vereinsmitglieder erfasste.

Früher wurden auf der Liste für jedes erworbene Getränk Striche in der Zeile des jeweiligen Mitglieds und in der Spalte des Getränks eingetragen. Regelmäßig zählte man die Striche, ermittelte so die Saldos der Mitglieder und erstellte eine neue Liste.

Der KM3003 ist die Weiterentwicklung der bisherigen Kühltischmanager (KM3000 bis KM3002). Der KM3002 basierte auf einem Raspberry Pi mit Touchscreen und NFC-Reader in einem 3D-gedruckten Gehäuse. Mitglieder wurden per Studentenausweis mit NFC identifiziert, und Käufe über den Touchscreen ausgewählt. Die Daten wurden in einer MySQL-Datenbank gespeichert, die jedoch nur negative Salden ermöglichte und so exakte Abrechnungen erzwang. Zudem gab es Verbesserungsbedarf in Codequalität, Zuverlässigkeit und Geschwindigkeit.

Der KM3003 erlaubt nun die Identifizierung von Mitgliedern und Produkten über das Scannen von Barcodes. Mitglieder werden über die Barcodes auf den Studentenausweisen erkannt, Produkte über EAN-Codes. Außerdem erlaubt er nun auch positive Saldos, sodass Mitglieder flexibel Guthaben einzahlen können.

Zusammengefasst ist der KM3003 ein Self-Checkout Point-of-Sale-Gerät mit Touchscreen und Barcode-Scanner.

2 Plattform und Systemarchitektur

Als Basis für den Kühltischmanager wurde ein Microsoft Surface 3 mit dem zugehörigen Dock verwendet. Das Surface 3 ist ein Tablet-Computer mit einem zuverlässigen kapazitiven Multi-Touchscreen und solider Treiberunterstützung unter Windows. Als Betriebssystem wird ein unverändertes Windows 11 Home verwendet, um eine gut getestete und stabile Basis zu bieten. Auch die Verfügbarkeit von Updates in der Zukunft wird so maximiert.

Weiterhin bieten die meisten Microsoft Surface Geräte einen Kioskmodus der spezielle Lösungen für Geräte mit festem und ortsunveränderlichem Verwendungszweck bietet. So optimiert der Kioskmodus das Ladeverhalten für Geräte, die permanent mit Strom versorgt werden, um die Akkulebensdauer

zu verlängern. Weiterhin kann der Zugang zu Anwendungen, die nicht dem gedachten Verwendungszweck entsprechen, beschränkt werden.

Zum scannen wird ein Freihandscanner (Modell DS9208) der Firma Zebra verwendet. Dieser wird per USB angeschlossen, meldet sich aber unter Windows, als ein über serielle Schnittstelle angebundenes Gerät. Dies erfolgt unter Verwendung des USB communications device class (USB CDC) Treibers.

Das Surface speichert keinerlei Daten. Hierfür wird eine externe MySQL-Datenbank verwendet, welche Nutzer, Produkte, Ein- bzw. Auszahlungen und Einkäufe enthält.

3 Software

Als Implementierungssprache wurde objektorientiertes Python3 verwendet. Python ist weit verbreitet und einfach zu lernen, was Wartungsarbeiten oder die Erweiterung mit neuen Features für zukünftige Mitglieder erleichtert.

Weiterhin können Python Anwendungen auf Linux und Windows Plattformen ausgeführt werden, sodass bei Bedarf auch das bisher verwendete Raspberry Pi mit externem Touchscreen weiter verwendet werden könnte.

Auch die Verfügbarkeit von bestehenden Bibliotheken für graphical user interface (GUI), Datenbankanbindung und serielle Schnittstellen ist optimal.

3.1 Verwendete Bibliotheken und Frameworks

3.1.1 Graphical User Interface

Bei der Auswahl eines GUI-Frameworks für Python-Projekte bieten PyQt, Tkinter und PySimpleGUI unterschiedliche Stärken und Einschränkungen in Bezug auf Lizenzmodell, Dokumentation und Architektur.

Tkinter und PyQt bieten Python Schnittstellen zu den externen GUI toolkits Tk und Qt. PySimpleGUI hingegen baut auf solchen toolkits auf und legt eine weitere Abstraktionsebene darüber um das Erstellen von GUIs über diese Toolkits hinweg zu vereinheitlichen und weiter zu vereinfachen.

PySimpleGUI unterliegt seit Version 5 einer projekteigenen Lizenz, welche die kostenlose Nutzung für Hobbyentwickler und Bildungszwecke erlaubt, sofern keine kommerzielle Nutzung stattfindet. Tkinter und PyQt sind GPL kompatibel bzw. direkt mit GPL lizenziert. Bei Tkinter und PyQt ist nicht

davon auszugehen, dass sich an der Lizenzierung etwas ändert. Bei PySimpleGUI hängen muss jedes Jahr wieder eine neue Hobbyisten- Lizenz über einen Account gelöst und wieder im Projekt eingepflegt werden. In Anbetracht der bisherigen Änderungen und dem zusätzlichen Arbeitsaufwand die Lizenz jährlich zu aktualisieren ist PySimpleGUI lizenztechnisch die schlechteste Wahl.

Die Dokumentation der drei Frameworks variiert stark in der Zielgruppe und Benutzerfreundlichkeit. PySimpleGUI ist stark auf Hobbyentwickler ausgelegt und verwendet viele Beispielprogramme, um Funktionen zu erklären. Tkinter und PyQt bieten eine standardisierte Dokumentation, die weniger auf Anfänger zugeschnitten ist, aber erfahrenen Entwicklern eine effizientes Nachschlagewerk bietet.

Architektonisch ähneln sich die Frameworks in der Grundlegenden Handhabung ihrer Komponenten. Alle genannten Frameworks arbeiten mit Elementen, welche durch Python Objekte repräsentiert werden. Diese Objekte haben Parameter und können durch die Nutzung von Hierarchien und Geometriemanagern positioniert werden.

Die Beispielerorientierte Dokumentation von PySimpleGUI ermöglicht schnelle erste Ergebnisse und macht es für Einsteiger sehr zugänglich. Aus diesem Grund wurde zu Beginn dieses Projektes PySimpleGUI als Framework ausgewählt, was sich jedoch schnell als Fehlentscheidung herausstellte. Die unstrukturierte Dokumentation wird schnell zum Hindernis und auch das Lizenzmodell steht, wie oben erklärt, PyQt und Tkinter nach. Für das Projekt KM3004 wird vorraussichtlich Tkinter verwendet.

3.1.2 Datenbank

Für die Anbindung einer MySQL Datenbank gibt es verschiedene Optionen welche verwendet werden können, die sich aber für diesen Anwendungsfall nur geringfügig unterscheiden. Der von Oracle selbst entwickelte MySQL Python Connector und PyMySQL sind beide PEP249 kompatibel, sodass sich deren API meist nur im Namen der Methoden unterscheiden. In diesem Projekt wurde PyMySQL verwendet, da ich Community-Lösungen gegenüber Angeboten von kommerziellen Anbietern bevorzuge. [3] [1]



Figure 1: The GUI of KM3003

3.2 Datenbankstruktur

Bei der Konzeption der Datenbank wurde speziell darauf geachtet Inkonsistenzen unmöglich zu machen und redundant gespeicherte Information zu verhindern. [2]

Die Datenbank besteht aus den Tabellen users, products, purchases und deposits.

durch die Verwendung von Fremdschlüsseln,

Bei der Konzeption der Datenbankstruktur wurde speziell darauf geachtet die Datenbank Normalformen, soweit es den Aufwand rechtfertigt, einzuhalten. Alle Felder sind atomar, sodass die **1NF!** (**1NF!**) erfüllt ist. Weiterhin nutzt die Datenbank keine zusammengesetzten Primärschlüssel, sodass auch die **2NF!** (**2NF!**) erfüllt ist.

jedes Nichtprimärattribut von allen also dem einen Schlüssenanteil abhängig ist und somit auch die zweite Normalform erfüllt ist. Die **3NF!** (**3NF!**) verlangt, dass keine Abhängigkeiten zwischen Nichtschlüsselanteilen bestehen. Dies wurde in der Tabelle purchases verletzt, da eine Abhängigkeit zwischen

price.then und date in Kombination mit der product.id besteht. Dies führt aber lediglich dazu, dass der Preis eines Produktes in einem Bestimmten Zeitraum für jeden Einkauf redundant gespeichert wird. Es kann hier nur zu Inkonsistenzen führen, wenn der Wert des Attributes price.then im Nachhinein geändert wird. Dies bliebe allerdings allerdings für den Betrieb folgenlos, da dieses Feld rein Statistische Zwecke erfüllt. Lösen könnte man dieses Problem, indem man eine neue Tabelle prices einführt, in welcher jeder Preis mit seinem Gültigkeitszeitraum nur einmal vorkommt.

Weiterhin wurde die **3NF!** in der Tabelle users verletzt.

3.3 Testing

Unit tests schreiben mit pyunit test Framework

4 Deployment

- deployment: surface, ansible for km3003 standalone cyclical restart windows task scheduler mysql db

5 Herausforderung bei der Implementierung

- wiederaufbau von Verbindungsabbrüchen (Datenbank und Scanner) zur Laufzeit

6 Ausblick und Verbesserungsvorschläge

UI Framework is kacke Sounds

7 Inline documetation anhängen

USB CDC USB communications device class

GUI graphical user interface

List of Figures

1	The GUI of KM3003	6
---	-----------------------------	---

References

- [1] *Mysql-Connector-Python: A Self-Contained Python Driver for Communicating with MySQL Servers, Using an API That Is Compliant with the Python Database API Specification v2.0 (PEP 249)*. Version 9.1.0. URL: <https://dev.mysql.com/doc/connector-python/en/> (visited on 11/20/2024).
- [2] *Normalisierung (Datenbank)*. In: *Wikipedia*. Nov. 19, 2024. URL: [https://de.wikipedia.org/w/index.php?title=Normalisierung_\(Datenbank\)&oldid=250491112](https://de.wikipedia.org/w/index.php?title=Normalisierung_(Datenbank)&oldid=250491112) (visited on 11/20/2024).
- [3] *PyMySQL: Pure Python MySQL Driver*. Version 1.1.1. URL: <https://pypi.org/project/PyMySQL/>.